

# LT1 ungraded demo practice

## EECS 3311 B | PrintDesk | September 29, 2026

Suggested time: 30 minutes; allow up to 35 minutes for the diagram.

Practise small-object validation, a supplied collaborator, protected snapshots, focused JUnit tests and a tiny UML explanation. This revisits L02-L05 and Labs 01-02 taught by September 25. It is a rehearsal, not the live test paper or starter, and carries no mark or required submission.

### 1. Open and predict - 5 minutes

Extract the whole ZIP. Open **EECS3311B\_LT1\_Ungraded\_PrintDesk\_Practice**, the folder containing README.md, run scripts and src/. The pinned JUnit 6.0.3 jar is cached in lib/. No network, account, API key or live model is needed to run the exercise.

```
java -version
javac -version
```

Both should identify JDK 21. Then run the route supported by your computer:

```
macOS/Linux or supported Git Bash:
bash run.sh compile
bash run.sh check
bash run.sh test
```

```
Supported Windows PowerShell:
./run.ps1 compile
./run.ps1 check
./run.ps1 test
```

**Expected starting point:** source compiles; of 9 supplied checks, 1 passes and 8 fail intentionally. These are unfinished-behaviour failures. Missing compiler, missing jar or checksum failure is a setup problem. JUnit output and ordinary Java check output are different evidence.

Follow the institution-supported setup if scripts are restricted. Do not change managed security settings. setup-junit is an optional repair tool; the included jar makes a download unnecessary. A TA-provided 6.0.3 jar can be selected with JUNIT\_JAR and must match the checksum.

Before editing, predict what PrintJob("P7", 3) and one submit call should do. Identify the field, constructor parameter, object receiving each call and state to protect.

# Complete the small design

## 2. Complete the behaviour - 12 minutes

A print desk accepts a validated job and sends one receipt to a collaborator. All data is synthetic; nothing is actually printed or transmitted. Edit only the TODOs in `src/PrintJob.java`, `src/PrintDesk.java` and `src/RecordingReceiptSink.java`. Preserve the public method signatures and supplied checks.

| Operation                        | Required behaviour                                                                                                                                                          |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>PrintJob(id, pages)</code> | Reject null/blank id and integer pages outside 1-10 with <code>IllegalArgumentException</code> . Store valid input exactly; preserve private final fields and read methods. |
| <code>PrintDesk(sink)</code>     | Reject null with <code>IllegalArgumentException</code> ; retain the supplied <code>ReceiptSink</code> .                                                                     |
| <code>submit(job)</code>         | Reject null before recording; for a valid job call the supplied sink exactly once.                                                                                          |
| <code>snapshot()</code>          | Return receipts in insertion order in an unmodifiable, detached list. A caller cannot edit it. Later records must not change an earlier snapshot.                           |

`ReceiptSink`, `Main` and the recorder's `accept` operation are provided. `List.copyOf(...)` is available for the protected snapshot. `PrintJob` has immutable fields and no setter, so sharing references to these jobs is safe in this exercise.

## 3. Test and explain - 8 minutes

Run check and test again. All nine supplied cases should pass after completion. Add **two focused Jupiter @Test methods** in `test/junit/StudentTests.java`, using fresh objects:

1. Submit two jobs; check exact receipt order and count.
2. Attempt to clear a snapshot; check rejection and confirm the recorder still contains its job.

Use real assertions and state expected results first. With these additions JUnit should discover 11 tests. The supplied wrapper alone does not demonstrate independent test design.

Draw `PrintDesk`, `ReceiptSink`, `RecordingReceiptSink` and `PrintJob` in `DESIGN.md`. Show interface realization and relevant associations/multiplicities. Include a text alternative; a hand drawing is fine. Explain why receiving a collaborator through a constructor does not imply exclusive lifecycle ownership.

# Check understanding

## The public cases

The nine supplied cases check: valid page endpoints; invalid page counts; null/blank identifiers; null collaborator; exactly one receipt; rejected null submission with no record; an unmodifiable snapshot; an old snapshot unchanged by later records; and separate recorder state.

Do not weaken or delete a check to make it pass. A green finite suite is useful evidence, not proof of every possible behaviour. Save actual output in results.txt if keeping a practice record.

## 4. Compare and rehearse - 5 minutes

Try the task before opening the separately supplied **Self\_Check\_Answers.md**. It provides the practice answers, explanation and two example tests. These are not live-test answers.

Explain one rejection case and one snapshot case aloud without assistance. Sketch the message order for a valid submission: which object receives submit, which receives accept and when the receipt is added? Explain why invalid input must be rejected before any receipt appears.

Allow up to 35 minutes if you want extra time for the diagram. This is a suggested practice budget, not a measured lab-computer pilot. There is no score, attendance credit or required submission for this demo.

## Practice and test rules

Begin with your own prediction and design explanation. If using assistance later while practising, record the files and verification in AI\_USE.md. The supervised Lab Test 1 prohibits generative AI under its issued rules.

Practice access does not make a resource permitted during the assessment. Follow the final Section B instructions for test rules, tasks, commands, permitted materials and submission. Use the LT1 preparation guide for the full revision map; this is one rehearsal and does not predict the exact questions.

## Files included

Source, scripts, all public checks, a blank learner-test class, DESIGN.md, AI\_USE.md, results.txt and the cached JUnit dependency are in the student starter. Self-check answers are supplied separately. The teaching team holds a runnable practice reference solution; no hidden grading or live assessment material is included in this starter.

**Course baseline:** Java 21 and JUnit 6.0.3. Existing Lab01/Lab02 script conventions are preserved. No later design-pattern, named SOLID, GUI, API or model requirement is introduced.